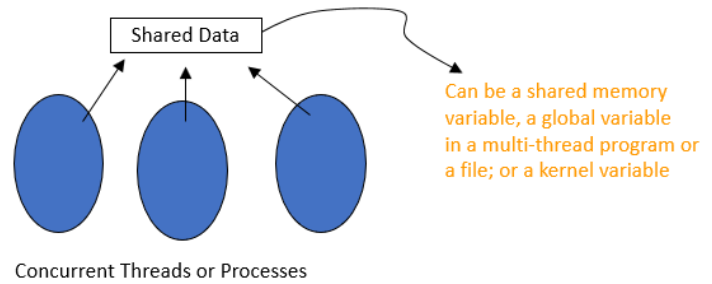


Chapter 5: Process Synchronization

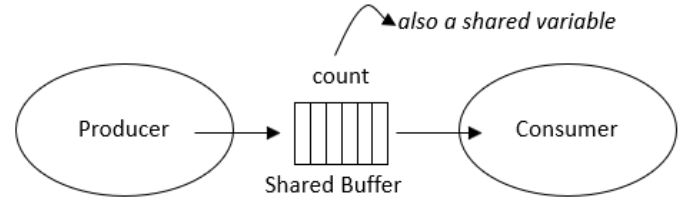
Background

- Concurrent access to shared data may result in data inconsistency
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes



Producer Consumer Problem Revisited

Suppose that we wanted to provide a solution to the consumer-producer problem that fills all the buffers. We can do so by having an integer count that keeps track of the number of full buffers. Initially, count is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.



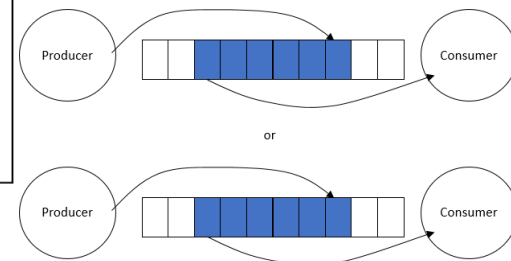
Producer and Consumer Code

PRODUCER

```
while (true) {  
    /* produce an item and  
    put in nextProduced */  
  
    while (count == BUFFER_SIZE)  
        ; // do nothing  
  
    buffer[in] = nextProduced;  
    in = (in + 1) % BUFFER_SIZE;  
    count++;  
}
```

CONSUMER

```
while (true) {  
    while (count == 0)  
        ; // do nothing  
  
    nextConsumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
    count--;  
  
    /* consume the item  
    in nextConsumed */  
}
```



a possible Problem: race condition

- Assume we had 5 items in the buffer
- Then:
 - Assume producer has just produced a new item and put it into buffer is about to increment the count.
 - Assume the consumer has just retrieved an item from buffer and is about to decrement the count.
 - Namely: Assume producer and consumer is now about to execute count++ and count-- statements.

Race Condition

- count++ could be implemented as

```
register1 = count  
register1 = register1 + 1  
count = register1
```
- count-- could be implemented as

```
register2 = count  
register2 = register2 - 1  
count = register2
```

Interleaved Execution sequence

- Consider this execution interleaving with "count = 5" initially:
S0: producer execute register1 = count {register1 = 5}
S1: producer execute register1 = register1 + 1 {register1 = 6}
S2: consumer execute register2 = count {register2 = 5}
S3: consumer execute register2 = register2 - 1 {register2 = 4}
S4: producer execute count = register1 {count = 6}
S5: consumer execute count = register2 {count = 4}

Programs and critical sections

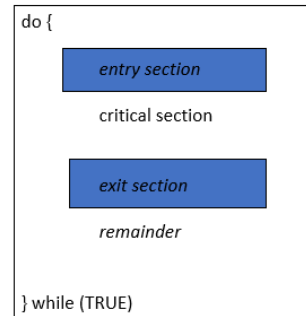
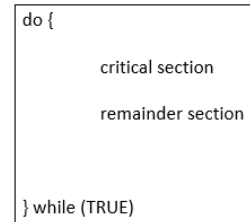
- The part of the program (process) that is accessing and changing shared data is called its critical section
- Considering a process:
 - It may be executing critical section code from time to time
 - It may be executing non critical section code (remainder section) other times.
 - We should not allow more than one process to be in their critical regions where they are manipulating the *same* shared data.

Solution to Critical-Section Problem

1. **Mutual Exclusion** - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
2. **Progress** - If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely
3. **Bounded Waiting** - A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

Structuring Programs

- The general way to do that is:



The general structure of a program

Entry section will allow only one process to enter and execute critical section code.

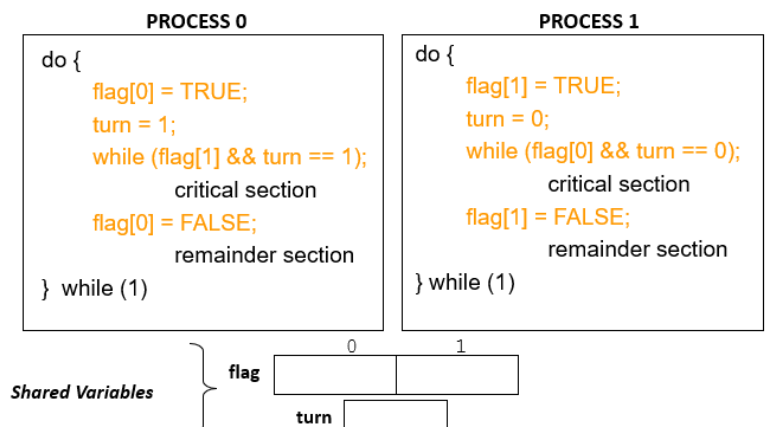
Peterson's Solution

- Two process solution
- Assume that the LOAD and STORE instructions are atomic; that is, cannot be interrupted.
- The two processes share two variables:
 - int turn;
 - Boolean flag[2]
- The variable turn indicates whose turn it is to enter the critical section.
- The flag array is used to indicate if a process is ready to enter the critical section. $\text{flag}[i] = \text{true}$ implies that process P_i is ready!

Algorithm for Process P_i

```
do {  
    entry section  
    critical section  
    exit section  
    remainder section  
} while (1)
```

Two processes executing concurrently



Bakery Algorithm in Process Synchronization

- The **Bakery algorithm** is one of the simplest known solutions to the mutual exclusion problem for the general case of N process. Bakery Algorithm is a critical section solution for N processes. The algorithm preserves the first come first serve property.
- Before entering its critical section, the process receives a number. Holder of the smallest number enters the critical section.
 - If processes P_i and P_j receive the same number

if $i < j$

P_i is served first;

else

P_j is served first.

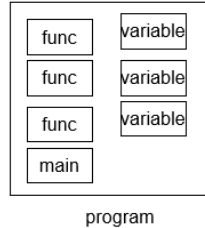
- The numbering scheme always generates numbers in increasing order of enumeration; i.e., 1, 2, 3, 3, 3, 3, 4, 5, ...

Algorithm Pseudocode –

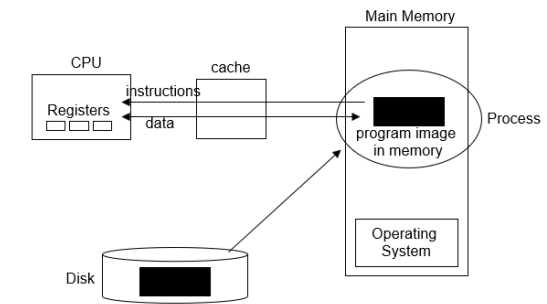
```
repeat
  choosing[i] := true;
  number[i] := max(number[0], number[1], ..., number[n - 1])+1;
  choosing[i] := false;
  for j := 0 to n - 1
    do begin
      while choosing[j] do no-op;
      while number[j] != 0
        and (number[j], j) < (number[i], i) do no-op;
    end;
  critical section
    number[i] := 0;
  remainder section
until false;
```

Memory Management

- When code is generated (or assembly program is written) we use memory addresses for variables, functions and branching/jumping.
- Those addresses can be physical or logical memory addresses.
- In very early systems they are just physical memory addresses.
 - A program has to be loaded to that address to run.
 - No relocation

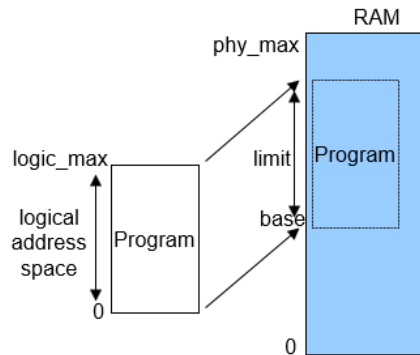


Background



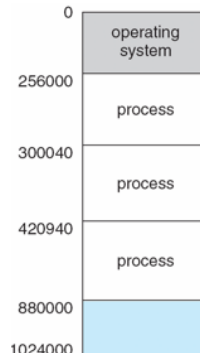
Logical address space concept

- We need logical address space concept, that is different than the physical RAM (main memory) addresses.
- A program uses logical addresses.
- Set of logical addresses used by the program is its logical address space
 - Logical address space can be, for example, $[0, \text{max_address}]$
- Logical address space has to be mapped somewhere in physical memory



Base and Limit Registers

A pair of **base** and **limit** registers define the address space of a process. A process should be accessing and using that range.



Protection and Relocation can be provided in this way.

also called Relocation Register

Each physical address should be in range $[\text{base}, \text{base} + \text{limit}]$

Logical vs. Physical Address Space

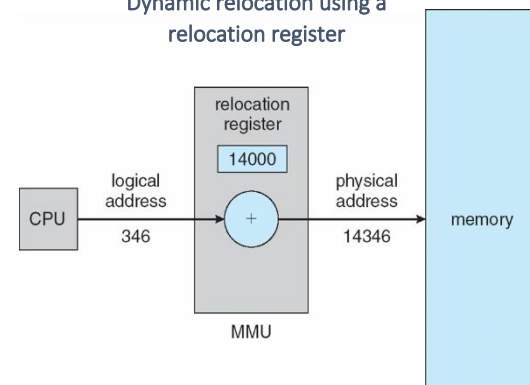
The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management

- Logical address** – generated by the CPU; also referred to as **virtual address**
- Physical address** – address seen by the memory unit

Memory-Management Unit (MMU)

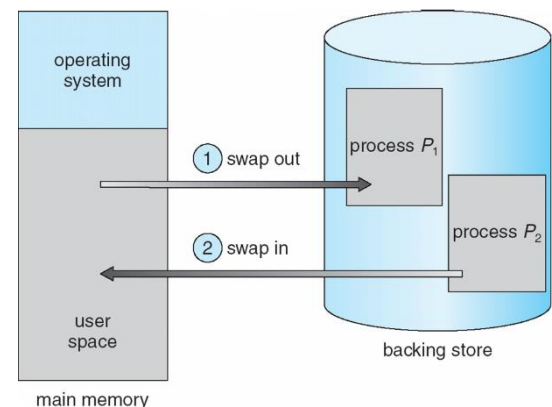
- Hardware device that maps logical (virtual) to physical address
- In MMU scheme, the value in the relocation register (i.e., base register) is added to every address generated by a user process at the time it is sent to memory
- The user program deals with *logical* addresses; it never sees the *real* physical addresses

Dynamic relocation using a relocation register



Swapping

- A process can be swapped temporarily out of memory to a backing store, and then brought back into memory for continued execution
- Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Modified versions of swapping are found on many systems (i.e., UNIX, Linux, and Windows)
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk



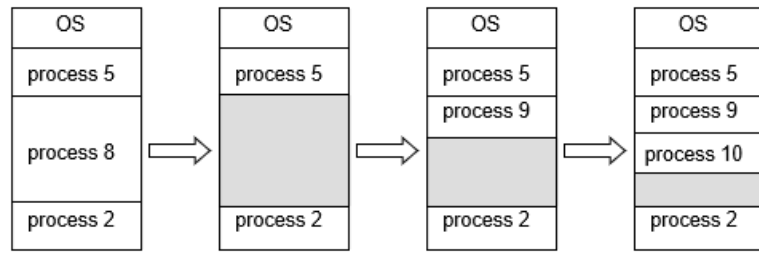
Basic Memory Allocation Strategies

3 basic main memory allocation strategies to processes

- 1) Contiguous allocation
- 2) Paging
- 3) Segmentation

Contiguous Allocation

- Main memory is partitioned usually into two partitions:
 - Resident operating system, usually held in low memory with interrupt vector
 - User processes then held in high memory
- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
 - Base register contains value of smallest physical address
 - Limit register contains range of logical addresses – each logical address must be less than the limit register
 - MMU maps logical addresses *dynamically*
- Multiple-partition allocation
 - Hole – block of available memory; holes of various size are scattered throughout memory
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it
 - Operating system maintains information about:
 - a) allocated partitions b) free partitions (hole)



Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes

- **First-fit:** Allocate the *first* hole that is big enough
- **Best-fit:** Allocate the *smallest* hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
- **Worst-fit:** Allocate the *largest* hole; must also search entire list
 - Produces the largest leftover hole

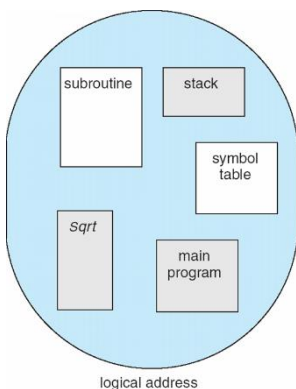
First-fit and best-fit better than worst-fit in terms of speed and storage utilization

Paging

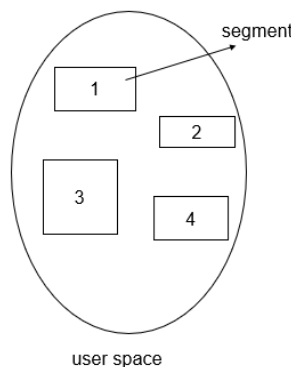
- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
 - Physical address space will also be **noncontiguous**.
- Divide physical memory into fixed-sized blocks called **frames** (size is power of 2, between 512 bytes and 8,192 bytes)
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size n pages, need to find n free frames and load program
- Set up a page table to translate logical to physical addresses
- Internal fragmentation

Segmentation

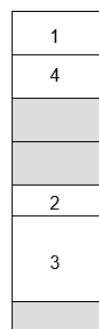
- Memory-management scheme that supports user view of memory
- A program is a collection of segments
 - A segment is a logical unit such as: main program, procedure, function, method, object, local variables, global variables, common block, stack, symbol table, arrays



User's View of a Program



Logical View of Segmentation

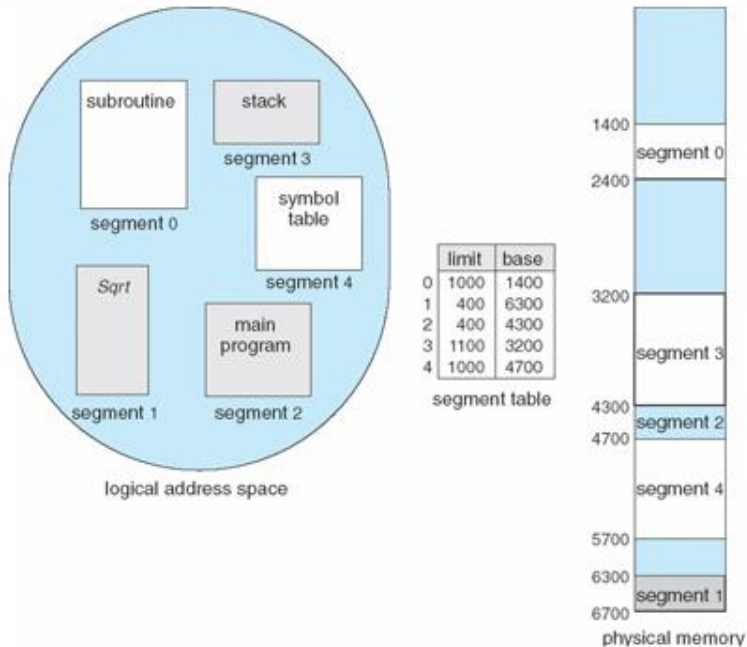


physical memory space

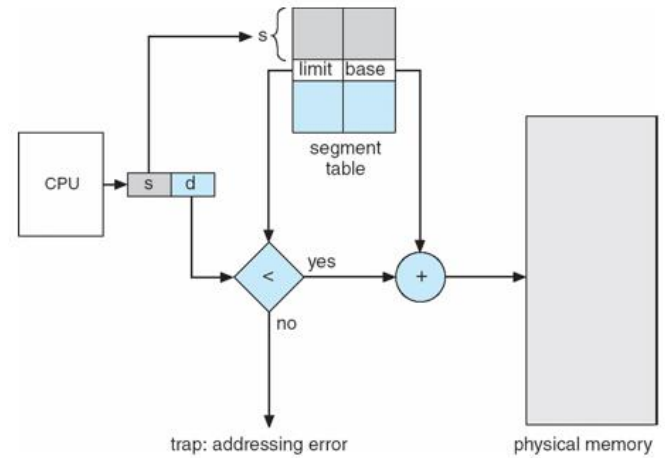
Segmentation Architecture

- Logical address consists of a two tuple:
<segment-number, offset>
- **Segment table** – maps two-dimensional logical addresses; each table entry has:
 - **base** – contains the starting physical address where the segments reside in memory
 - **limit** – specifies the length of the segment
 - **Segment-table base register (STBR)** points to the segment table's location in memory
- **Segment-table length register (STLR)** indicates number of segments used by a program;

Example of Segmentation



Segmentation Hardware

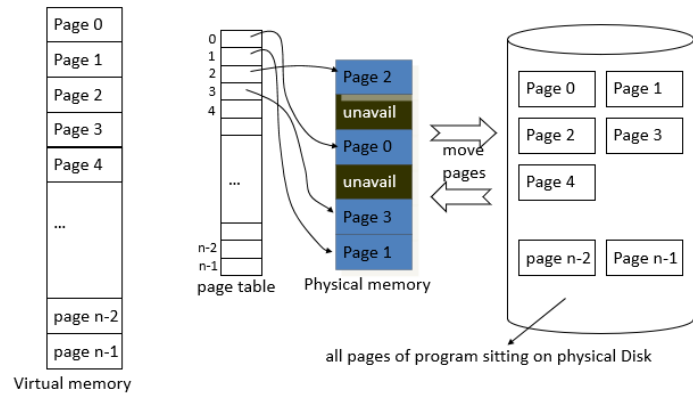


Virtual Memory

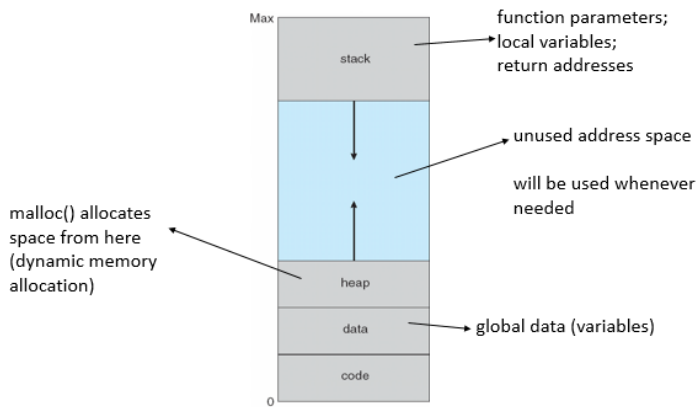
Background

- **Virtual memory** – program uses virtual memory which can be partially loaded into physical memory
- **Benefits:**
 - Only part of the program needs to be in memory for execution
 - more concurrent programs
 - Logical address space can therefore be much larger than physical address space
 - execute programs larger than RAM size
 - Easy sharing of address spaces by several processes
 - Library or a memory segment can be shared
 - Allows for more efficient process creation

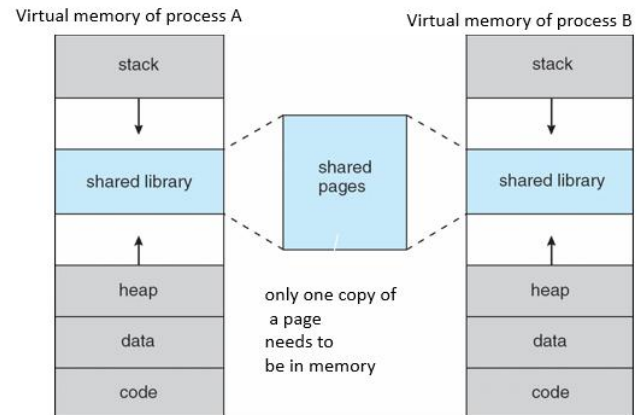
Virtual Memory That is Larger Than Physical Memory



A typical virtual-address space layout of a process



Shared Library Using Virtual Memory



Implementing Virtual Memory

Virtual memory can be implemented via:

- **Demand paging:** Bring pages into memory when they are used, i.e. allocate memory for pages when they are used
- **Demand segmentation:** Bring segments into memory when they are used, i.e. allocate memory for segments when they are used.

Policies for Paging/ Segmentation

- **Fetch Strategies**
 - When should a page or segment be brought into primary memory from secondary (disk) storage?
 - Demand Fetch
 - Anticipatory Fetch
- **Placement Strategies**
 - where is it to be put?
 - Paging - trivial
 - Segmentation - significant problem
- **Replacement Strategies**
 - Which page/segment should be replaced if there is not enough room for a required page/segment?

Demand Paging

- Bring a page into memory only when it is needed
 - Less I/O needed
 - Less memory needed
 - Faster response
 - More users
- Page is needed $\Rightarrow$ reference to it
 - invalid reference (page is not in used portion of address space) $\Rightarrow$ abort
 - not-in-memory $\Rightarrow$ bring to memory
 - **Pager** never brings a page into memory unless page will be needed

Valid-Invalid Bit

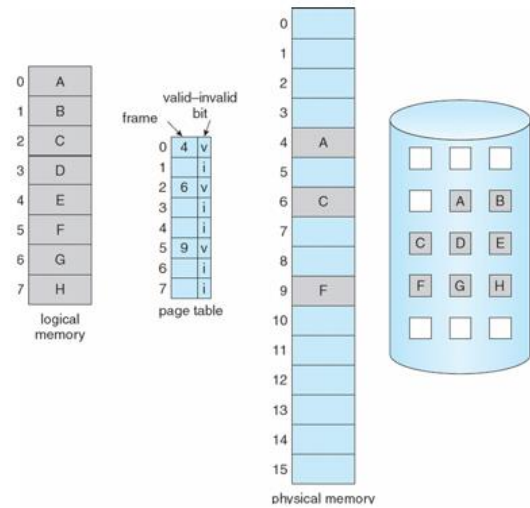
- With each **page table entry** a **valid-invalid bit** is associated (**v** $\Rightarrow$ in-memory, **i** $\Rightarrow$ not-in-memory)
- Initially valid-invalid bit is set to **i** on all entries
- Example of a page table snapshot:

Frame #	valid-invalid bit
	v
	v
	v
	v
	i
....	
	i
	i

page table

- During address translation, if valid-invalid bit in page table entry is **i** $\Rightarrow$ page fault

Page Table When Some Pages Are Not in Main Memory



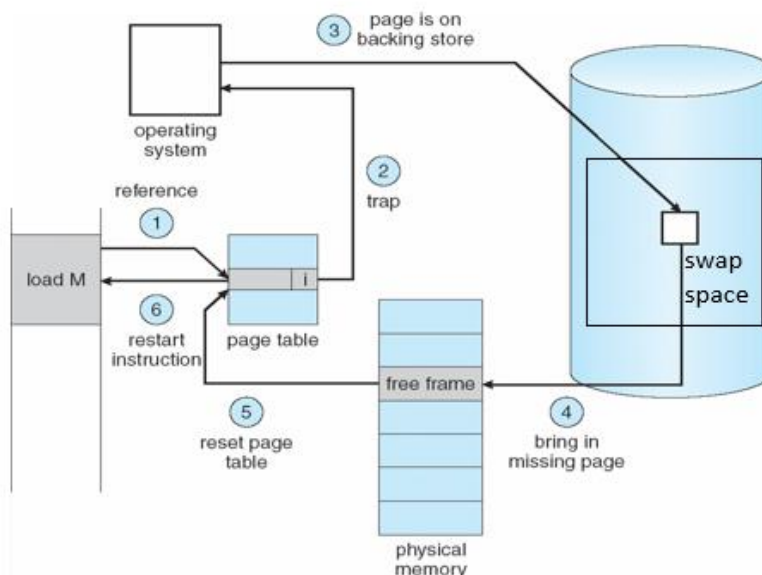
Page Fault

- When CPU makes a memory reference (i.e. page reference), HW consults the page table. If entry is invalid, then exception occurs and kernel gets executed.

Kernel handling such as case:

- Kernel looks at another table to decide:
 - Invalid reference (page is in *unused* portion of address space) $\rightarrow$ Abort
 - Just not in memory (page is in used portion, but not in RAM) $\rightarrow$ Page Fault
 - Get empty frame
(we may need to remove a page; if removed page is modified, we need **disk I/O** to swap it out)
 - Swap page into frame
(we need **disk I/O**)
 - Reset tables (install mapping into page table)
 - Set validation bit = **v**
 - Restart the instruction that caused the page fault
- If page fault occurs when trying to fetch an instruction, fetch the instruction again after bringing the page in.
 - If page fault occurs while we are executing an instruction: Restart the instruction after bringing the page in.

Steps in Handling a Page Fault



What happens if there is no free frame?

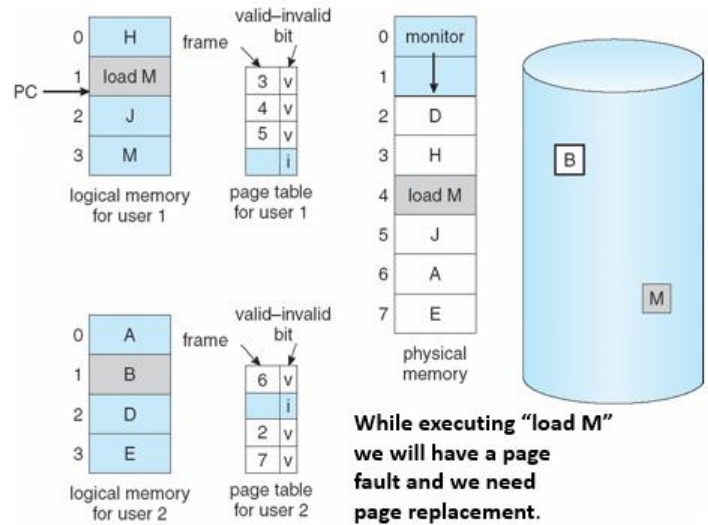
- Page replacement – find some page in memory, but not really in use, swap it out
 - Algorithm ? Which page should be remove?
 - performance – want an algorithm which will result in minimum number of page faults
 - With page replacement, same page may be brought into memory several times

Basic Page Replacement

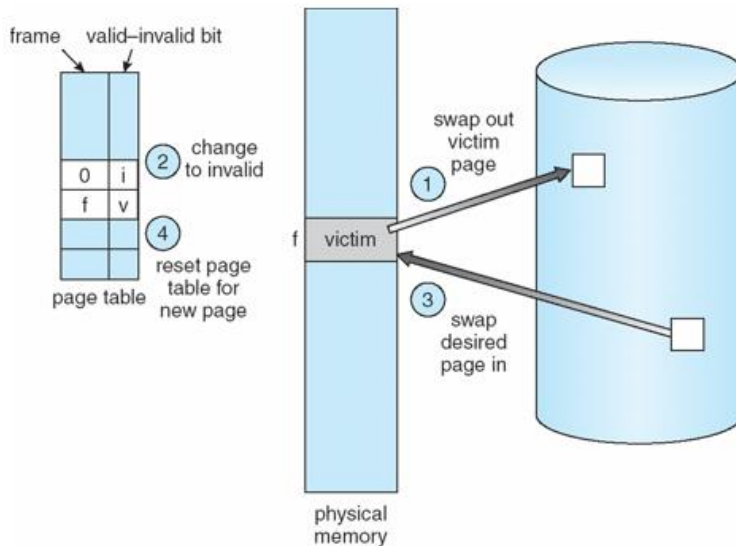
Steps performed by OS while replacing a page upon a page fault:

- Find the location of the desired page on disk
- Find a free frame:
 - If there is a free frame, use it
 - If there is no free frame, use a page replacement algorithm to select a **victim** frame; if the victim page is modified, write it back to disk.
- Bring the desired page into the (new) free frame; update the page and frame tables
- Restart the process

Need For Page Replacement



Page Replacement



Virtual Machines

Overview

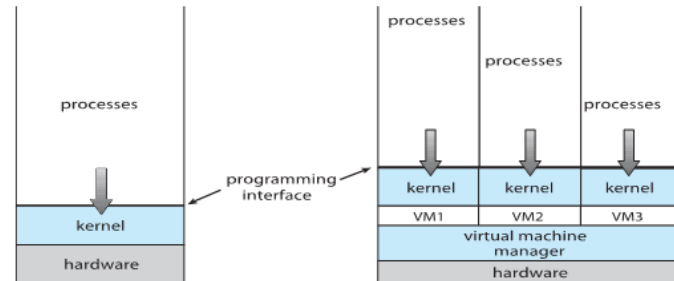
- Fundamental idea – abstract hardware of a single computer into several different execution environments
 - Similar to layered approach
 - But layer creates virtual system (virtual machine, or VM) on which operation systems or applications can run
- Several components
 - Host – underlying hardware system
 - Virtual machine manager (VMM) or hypervisor – creates and runs virtual machines by providing interface that is *identical* to the host
 - Guest – process provided with virtual copy of the host
 - Usually an operating system
- Single physical machine can run multiple operating systems concurrently, each in its own virtual machine

WHAT IS A VIRTUAL MACHINE MANAGER?

ALSO CALLED A HYPERVISOR

IT'S A TYPE OF SOFTWARE THAT ALLOWS US TO RUN AN OPERATING SYSTEM WITHIN ANOTHER OPERATING SYSTEM

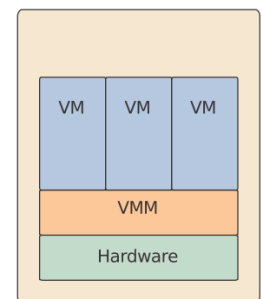
System Models



TYPE1 vs TYPE2 hypervisors

Type 1 hypervisors:

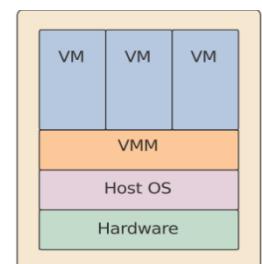
- It works directly on the hardware of the host and can monitor operating systems that run above the hypervisor.
- Completely independent from the Operating System.
- A major advantage is that any problems in one virtual machine or guest operating system do not affect the other guest operating systems running on the hypervisor.
- Examples:**
 - VMware ESXi Server
 - Microsoft Hyper-V
 - Citrix/Xen Server



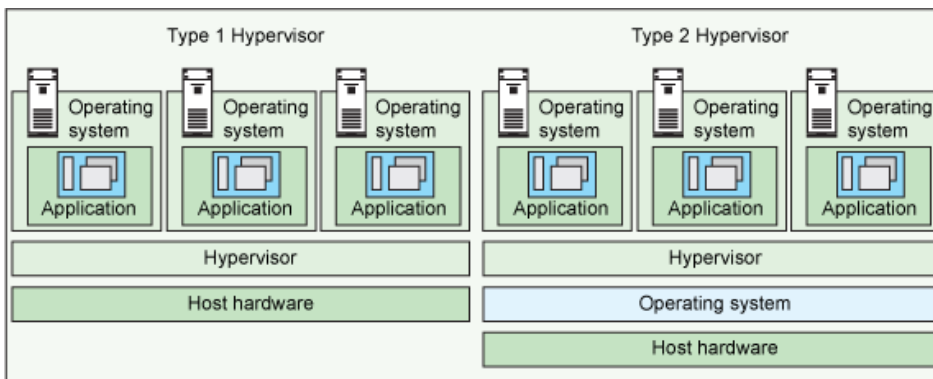
Hypervisor Type I

Type 2 hypervisors:

- Hypervisor is installed on an operating system and then supports other operating systems above it.
- It is completely dependent on host Operating System for its operations.
- While having a base operating system allows better specification of policies, any problems in the base operating system affects the entire system as well even if the hypervisor running above the base OS is secure.
- Examples: Vmware, Virtual Box.



Hypervisor Type II



Benefits and Features

- Host system protected from VMs, VMs protected from each other
 - i.e. A virus less likely to spread
 - Sharing is provided though via shared file system volume, network communication
- Freeze, suspend, running VM
 - Then can move or copy somewhere else and resume
 - Snapshot of a given state, able to restore back to that state
 - Some VMMs allow multiple snapshots per VM
 - Clone by creating copy and running both original and copy

- Great for OS research, better system development efficiency
- Run multiple, different OS on a single machine
 - Consolidation, app dev, ...
- Templating – create an OS + application VM, provide it to customers, use it to create multiple instances of that combination
- Live migration – move a running VM from one host to another!
 - No interruption of user access
- All those features taken together -> cloud computing
 - Using APIs, programs tell cloud infrastructure (servers, networking, storage) to create new guests, VMs, virtual desktops

Building Blocks

- Generally difficult to provide an *exact* duplicate of underlying machine
 - Especially if only dual-mode operation available on CPU
 - But getting easier over time as CPU features and support for VMM improves
 - Most VMMs implement virtual CPU (VCPU) to represent state of CPU per guest as guest believes it to be
 - ✓ When guest context switched onto CPU by VMM, information from VCPU loaded and stored

Building Block – Trap and Emulate

- Dual mode CPU means guest executes in user mode
 - Kernel runs in kernel mode
 - Not safe to let guest kernel run in kernel mode too
 - So VM needs two modes – virtual user mode and virtual kernel mode
 - Both of which run in real user mode
 - Actions in guest that usually cause switch to kernel mode must cause switch to virtual kernel mode
- **How does switch from virtual user mode to virtual kernel mode occur?**
 - Attempting a privileged instruction in user mode causes an error -> trap
 - VMM gains control, analyzes error, executes operation as attempted by guest
 - Returns control to guest in user mode
 - Known as trap-and-emulate
 - Most virtualization products use this at least in part
- User mode code in guest runs at same speed as if not a guest
- But kernel mode privilege mode code runs slower due to trap-and-emulate
 - Especially a problem when multiple guests running, each needing trap-and-emulate
- CPUs adding hardware support, mode CPU modes to improve virtualization performance

